RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

CS1003
Programming in C

Module - 2

Content of Module-2

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Data concepts in C:
  - Constants, Variables - Variables as named memory locations
  - Rules for Constructing Variables, understanding different Data Types - range, size, and appropriate usage,
  - C Keywords, format specifiers - %d, %c, %ud, %l, %f
  - Uses and Common Errors associated with them, Instruction, Pre-processor Directive, Macros.

- Operator in C:
  - Arithmetic Operators - Hierarchy of operations, Precedence
  - Relational Operators - (>, <, >=, <=, !=, ==)
  - Logical Operators - Logical AND (&&), Logical OR (||) Logical Not (!)
  - Conditional Operators - (?:),
  - Bit-wise Operators - (&, |, ^, ~, <<, >>),
  - Assignment Operators.
  - Type Conversion and Type Casting.

Constants

- Constant is a value that cannot be altered by the program during its execution.
- Used to represent fixed values that are intended to remain unchanged throughout the execution of the program

- Types of Constants
  - **Literal Constants:** Direct values in the code, such as numbers or characters.
    - Integer Literal: ‘10’,’-5’.
    - Floating Point Literal: ‘3.14’, ‘-0.678.
    - Character Literal: ‘a’,’z’.
    - String Literal:”RVUniversity”, “Hello World”

Variable

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- It is a storage location identified by a name (an identifier) that holds a value that can be modified during program execution.
- It must be declared before they can be used, specifying the type of data they will store.
- Types of Variable
  - Local Variable: Declared inside a function or block and can only be accessed within that function or block.
  - Global Variable: Declared outside of all functions and can be accessed from any function within the same file or other files

Variable

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

```
#include <stdio.h>
int globalVar = 5; // Global variable
void exampleFunction() {
    int localVar = 10; // Local variable
    printf("Local Variable: %d\n", localVar);
}
int main() {
    int num = 20; // Local variable in main function
    printf("Global Variable: %d\n", globalVar);
printf("Local Variable in main: %d\n", num);
 exampleFunction();
 num = 30; // Modify local variable

 printf("Modified Local Variable in main: %d\n", num);
  return 0;
}
```

Rules for Constructing Variables

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- These rules ensure that variable names are valid and meaningful within the context of the C language
- Start with a Letter or Underscore:
  - Variable names must begin with a letter (either uppercase or lowercase) or an underscore(_)
  - Example: int count; float _value;
- Followed by Letters, Digits, or Underscores:
  - Example: int age1,double temperature_value;
- Cannot Start with a Digit: Example: int 1stPlace;
- Case-Sensitive: Example: int totalAmount; and int TotalAmount; are different variables
- Cannot Be a Reserved Keyword: Example: int int; is invalid because int is a keyword.
- No Special Characters Except Underscores: Example: int score$; is invalid.
- Length Limits: While C does not specify a strict limit on variable name length, many compilers have practical limits. However, it is advisable to keep names reasonably short and meaningful for better readability.

Understanding different Data Types - range, size, and appropriate usage

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- C data types are defined as the data storage format that a variable can store data to perform a specific operation.
- Datatypes are used to define a variable before to use in a program
- Uses
  - Identify the type of a variable when it declared.
  - Identify the type of the return value of a function
  - Identify the type of a parameter expected by a function

Why Data Types?

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Every variable must have a type
- Every variable should get some space in the memory
- Help the compiler decide how many bits should be reserved for a variable

- Primary Data Types: These are fundamental or basic **data types in C** namely

1. int
2. float
3. character
4. void

Integer Data type

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Integers are used to store whole numbers
- Unsigned -> means positive number if no sign i.e. +
- Signed -> means positive or negative number with sign i.e. +/-

Type	Size(bytes)	Range	Format Specifier
int or signed int	2	-32,768 to +32767	%d
unsigned int	2	0 to 65535	%d
long int or signed long int	4	-2,147,483,648 to +2,147,483,647	%ld
unsigned long int	4	0 to 4,294,967,295	%lu

Floating Point/Real Data type

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Floating types are used to store real number

Type	Size(bytes)	Range	Format Specifiers
float	4	$3.4X10^{-38}$ to $3.4X10^{+38}$	%f
double	8	$1.7X10^{-308}$ to $1.7X10^{+308}$	%lf
long double	10	$3.4X10^{-4932}$ to $3.4X10^{+4932}$	%Lf

Character Data Type

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Character types are used to store characters value

Type	Size(bytes)	Range	Format Specifiers
char or signed char	1	-128 to 127	%c
unsigned char	1	0 to 255	%c

Void Data Type

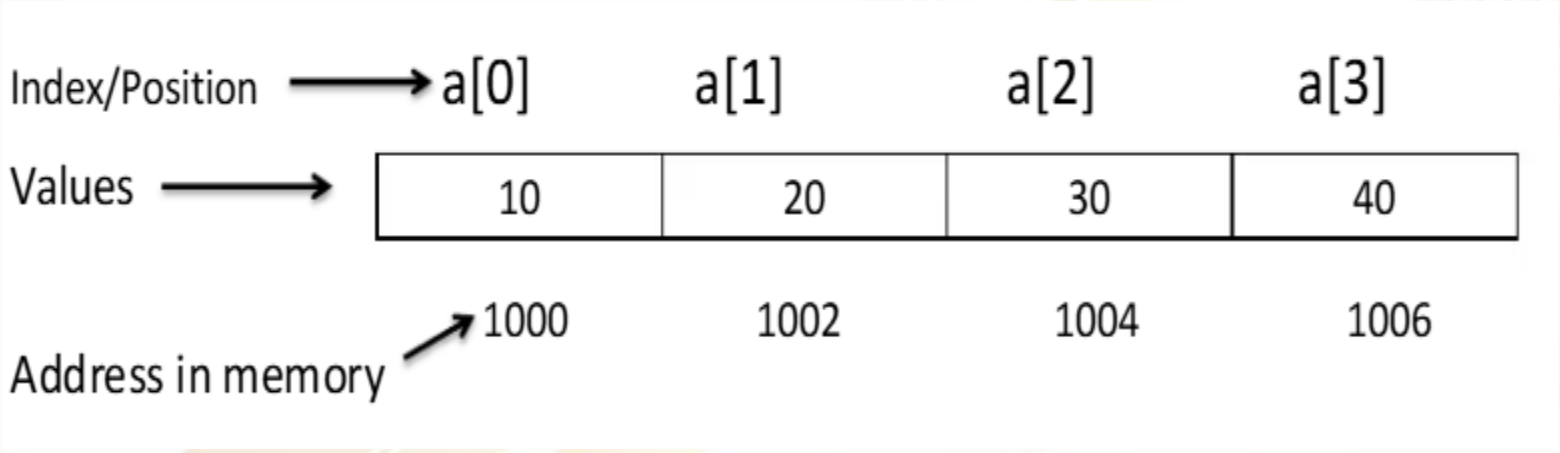
RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Void type means no value.
- This is usually used to specify the type of functions which returns nothing by defining in front of main() or any other function and as blank arguments within functional brackets.

- For example
- Void main()
- Void main(void)

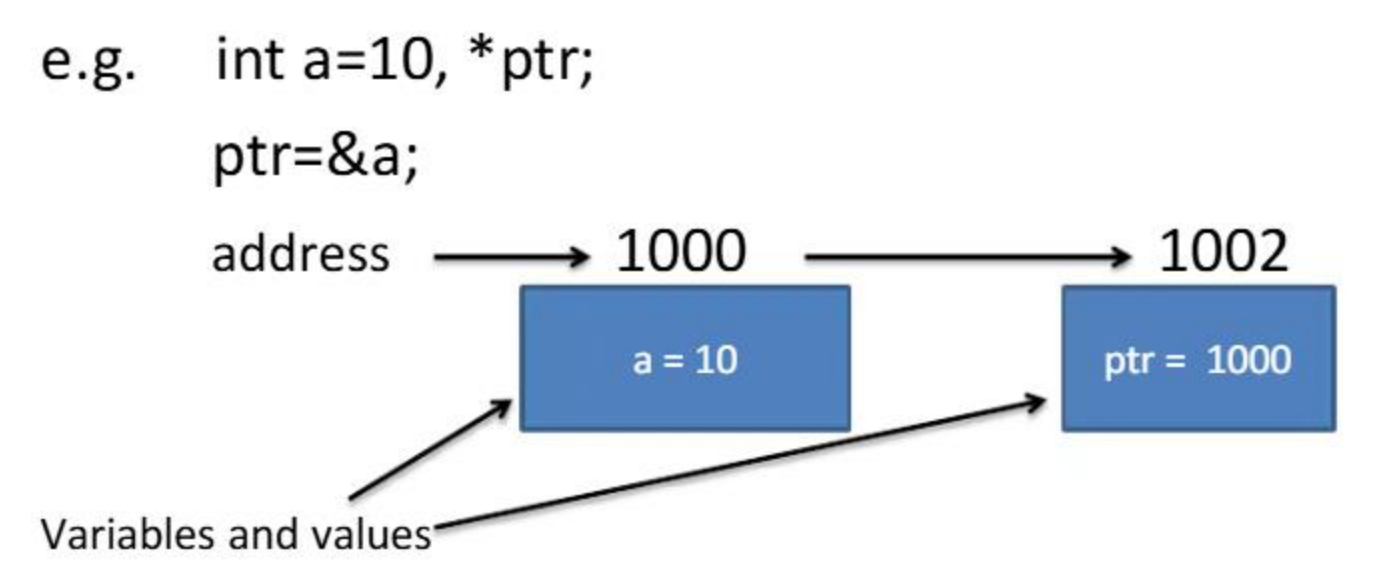
Derived Data Types

- Derived data types are types that are constructed from the basic (or primitive) data types.
- These derived types allow you to create complex data structures that can manage collections of data more effectively.
- Types
  - Arrays: Collection of items stored at continuous memory location
  - Example: int a[4]={10,20,30,40};



Derived Data Type cont..

- Pointers: Pointers are symbolic representation of addresses.



User-Defined Data Types

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- User-defined data types are data types that you define yourself, as opposed to the built-in primitive data types like int, char, and float.
- User-defined data types allow you to create complex data structures that better represent real-world entities and manage data in a more organized manner
- structure
- union
- enum
- typedef

Data Type	Size	Range (Typical)	Usage
`char`	1 byte (8 bits)	Signed: -128 to 127 Unsigned: 0 to 255	Store single characters or small integers.
`short`	2 bytes (16 bits)	Signed: -32,768 to 32,767 Unsigned: 0 to 65,535	Store smaller integers.
`int`	4 bytes (32 bits)	Signed: -2,147,483,648 to 2,147,483,647 Unsigned: 0 to 4,294,967,295	General-purpose integer storage.
`long`	4 bytes (32 bits) or 8 bytes (64 bits)	Signed (32-bit): -2,147,483,648 to 2,147,483,647 Unsigned (32-bit): 0 to 4,294,967,295 Signed (64-bit): -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 Unsigned (64-bit): 0 to 18,446,744,073,709,551,615	Larger integer storage when `int` is insufficient.
`long long`	8 bytes (64 bits)	Signed: -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 Unsigned: 0 to 18,446,744,073,709,551,615	Very large integer storage.
`float`	4 bytes (32 bits)	±1.7E−38 to ±3.4E+38	Single precision floating-point numbers.
`double`	8 bytes (64 bits)	±1.7E−308 to ±1.7E+308	Double precision floating-point numbers.
`long double`	8 bytes (64 bits), 12 bytes (96 bits), or 16 bytes (128 bits)	Typically larger than `double`, varies by implementation	Extended precision floating-point numbers.
`_Bool` (C99)	1 byte (8 bits)	`true` or `false`	Boolean values.

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Derived Data Type	Definition	Example	Usage
Array	Collection of elements of the same type stored in contiguous memory locations.	`int arr[10];`	To store multiple values of the same type.
Structure (`struct`)	User-defined type that groups variables of different types under a single name.	`struct Person { char name[50]; int age; };`	To group related variables together.
Union (`union`)	User-defined type where all members share the same memory location.	`union Data { int i; float f; };`	To store different types of data in the same memory location.
Enumeration (`enum`)	User-defined type consisting of integral constants.	`enum Weekday { MONDAY, TUESDAY, WEDNESDAY };`	To define a set of named integer constants.
Pointer	Variable that stores the address of another variable.	`int *ptr;`	To reference and manipulate memory addresses.

Keywords in C

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Keywords are reserved words that have special meanings and cannot be used for identifiers (e.g., variable names, function names).
- They are part of the C language syntax and are predefined by the language specification.
- Each keyword serves a specific purpose in C, defining the language's structure and control flow.

Keyword	Description
auto	Specifies automatic storage duration (default storage class for local variables).
break	Exits from a loop or a switch statement.
case	Defines a branch in a switch statement.
char	Declares a variable of character type.
const	Specifies that the value of a variable cannot be changed
continue	Skips the current iteration of a loop and proceeds to the next iteration.
default	Specifies the default case in a switch statement.
do	Initiates a do-while loop, which executes a block of code at least once and then repeats based on a condition.
double	Declares a variable of double precision floating-point type.
else	Specifies the block of code to execute if the condition in an if statement is false.
enum	Defines an enumeration, a set of named integer constants.
extern	Declares a variable or function that is defined in another file or scope.
float	Declares a variable of floating-point type.
for	Initiates a for loop, which repeats a block of code a specified number of times.

Keyword	Description
goto	Transfers control to a labeled statement within the same function.
if	Starts a conditional statement that executes a block of code if a specified condition is true.
int	Declares a variable of integer type.
long	Declares a variable of long integer type.
register	Suggests that the variable should be stored in a CPU register for faster access (note: this is a suggestion, not a guarantee).
return	Exits from a function and optionally returns a value.
short	Declares a variable of short integer type.
signed	Specifies a signed integer type, which can hold both positive and negative values.
sizeof	Returns the size, in bytes, of a variable or data type.
static	Specifies that the variable or function has internal linkage or maintains its value between function calls.
struct	Defines a structure, a composite data type that groups variables of different types.
switch	Allows multi-way branching based on the value of an expression.
typedef	Creates an alias for a data type.
union	Defines a union, a data type that can hold different types of data but only one at a time.

Keyword	Description
unsigned	Specifies an unsigned integer type, which can only hold non-negative values.
void	Specifies that a function does not return a value or indicates a generic pointer type.
volatile	Indicates that a variable may be changed by external factors (e.g., hardware) and should not be optimized by the compiler.
while	Initiates a while loop, which repeats a block of code as long as a specified condition is true.

C format specifiers - %d, %c, %ud, %l, %f, Uses and

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- format specifiers are used in functions like printf and scanf to define how data should be formatted when output or input is performed
- Format Specifiers(%d)
  - Use: Formats an int (signed integer) for output.
  - Example:
```
int num = 42;
printf("%d", num); // Output: 42
```
  - Using %d with a type other than int can lead to undefined behavior.
  - For instance, using %d with unsigned int can result in incorrect output.

- Format Specifier(%c)
- **Use**: Formats a single character for output.
- Example: char ch = 'A';

  printf("%c", ch); // Output: A
- Using %c with a non-character type, such as an int, without explicit casting, can produce unexpected results.

- Format specifier(%u)
- Use: Formats an unsigned int for output.
- Example: unsigned int num = 42;

  printf("%u", num); // Output: 42
- Using %u with a signed integer or other non-unsigned types can cause incorrect output or undefined behavior

- Format Specifier(%ld)
- **Use**: Formats a long int (signed long integer) for output.
- Example: long int num = 123456789L;

  printf("%ld", num); // Output: 123456789
- Using %ld with int or short can cause incorrect output. Ensure the type matches the format specifier.
- Format specifier(%f)
- Use: Formats a float or double for output. When using %f, it assumes a double argument due to default argument promotion
- Example: float num = 3.14f;

  printf("%f", num); // Output: 3.140000
- Using %f with types other than float or double can result in incorrect output or undefined behavior. Also, note that %f will print a float with six decimal places by default.

Format Specifier Common Errors associated with them

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

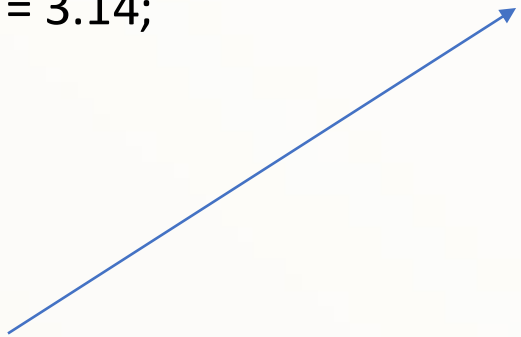
- **Type Mismatch**:
  - Using a format specifier that does not match the type of the argument leads to incorrect output or undefined behavior.
  - For example, using %d for a float or %f for an int is incorrect.
- Precision and width Misuse
  - For floating-point numbers, if you need to control the number of decimal places, use %.nf where n is the number of decimal places.
  - For example, %.2f formats a float to two decimal places.
- Unsigned Vs Signed Confusion
  - Using %u for signed integers or %d for unsigned integers can lead to confusing and incorrect output.
  - Ensure you match the specifier to the type (e.g., %u for unsigned int, %d for int).
- Long and short Modifiers
  - When using %ld, %lld, %hd, etc., ensure that the variable type matches the specifier.
  - %ld should be used for long int, %lld for long long int, and %hd for short int.

- Floating point precision
  - The %f specifier shows six decimal places by default.
  - If more precision is needed or if you want to round off to a specific number of decimal places, adjust accordingly.
- Proper use of format specifiers is essential for accurate and predictable behavior in C programs.
- Always match your format specifiers with the type of the variables being used, and double-check to avoid common pitfalls

Instructions in C

- **Instructions** in C are the fundamental building blocks of a C program.
  - They include statements that perform actions such as

1. Variable declarations: Ex: int a; float b;
2. Assignments: Ex: a = 10; b = 3.14;
3. Control flow operations. :Ex



Pre-Processor Directives in C

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Pre-processor directives are instructions processed by the pre-processor before actual compilation begins.
- Start with a # symbol and are used to control the compilation process.
- Some common pre-processor directives include:
- #include: Includes the contents of a file in the source code.
  - #include <stdio.h> // Includes the standard input/output library
- #define: Defines macros or constants. This directive replaces all instances of the macro with its definition before compilation.
  - #define PI 3.14159 // Defines a constant PI with the value 3.14159
- #undef: Undefines a macro, removing its definition.
  - #undef PI // Undefines the macro PI

Pre-Processor Directives in C

- #ifdef and #ifndef: Checks if a macro is defined or not defined, respectively. Useful for conditional compilation.
  - #ifdef DEBUG printf("Debug mode is enabled\n"); #endif
- #if, #elif, and #else: Conditional compilation based on compile-time constants.

```
#if defined(DEBUG) && (DEBUG > 1)
        printf("Debug level 2 or higher\n");
#else
        printf("Debug level 1 or not defined\n"); #endif
```

- #endif: Ends an #if, #ifdef, or #ifndef block.
- #error: Generates a compilation error with a specified message.
  - #error "This is an error message”
- #pragma: Provides additional information to the compiler; usage is compiler-specific.
  - #pragma once // Ensures the header file is included only once

Macros in C

- Macros are a type of pre-processor directive that allows you to define code snippets or constants that can be reused throughout your program.
- Macros are defined using #define and can be simple constants or more complex code blocks.

- Object-like Macros: Define constants or values.

```
#define MAX_SIZE 100 // Defines MAX_SIZE as 100
```

- **Function-like Macros**: Define code snippets that can take arguments.

```
#define SQUARE(x) ((x) * (x)) // Defines a macro to compute the square of a number
```

Ex: int result = SQUARE(5); // Expands to ((5) * (5)) and evaluates to 25

Pre defined Macros

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Macro	Value
__DATE__	A string containing the current date.
__FILE__	A string containing the file name.
__LINE__	An integer representing the current line number.
__STDC__	If follows ANSI standard C, then the value is a nonzero integer.
__TIME__	A string containing the current time.

Pre defined Macros_ Examples

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

```
#include<stdio.h>
int main()
{
printf("Date is %s\n",__DATE__);
printf("Time is %s\n",__TIME__);
printf("File is %s\n",__FILE__);
printf("Line is %d\n",__LINE__);
printf("ANSI is %d\n",__STDC__);
return 0;
}
```

Operators in C: Arithmetic operators

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Used to perform mathematical operations.
- Understanding their hierarchy, associativity, and precedence is crucial for writing correct expressions and avoiding errors.
- Operator precedence determines the order in which operators are evaluated in an expression

1. Parentheses ():Operations inside parentheses are performed first.Example: (2 + 3) * 4 evaluates to 5 * 4, which is 20.
2. Unary Operators:
    1. Unary plus + and unary minus - (e.g., -5, +3)
    2. Increment ++ (e.g., x++, ++x)
    3. Decrement -- (e.g., x--, --x)
    4. These are evaluated after parentheses but before other arithmetic operations

Arithmetic operators cont..

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

3. Multiplication, Division, and Modulus *, /, %:
   - Same level of precedence and are evaluated from left to right.
   - Example: 10 / 2 * 3 evaluates to 5 * 3, which is 15.

4. Addition and Subtraction +, -
   - Same level of precedence and are evaluated from left to right.
   - Example: 5 + 3 - 2 evaluates to 8 - 2, which is 6.

Associativity

- Associativity determines the order in which operators of the same precedence are evaluated:

1. Left-to-Right Associativity:
   1. Operators like *, /, %, +, and - are evaluated from left to right.
   2. Example: 6 / 2 * 3 evaluates as (6 / 2) * 3, which is 3 * 3, giving 9.

2. Right-to-Left Associativity:

- Unary operators like ++, --, and unary + and - have right-to-left associativity.
- Example: --x + 5 first decrements x and then adds 5.

Operator	Description	Precedence	Associativity
()	Parentheses	Highest	N/A
++ (post)	Post-increment	High	Left-to-Right
-- (post)	Post-decrement	High	Left-to-Right
++ (pre)	Pre-increment	High	Right-to-Left
-- (pre)	Pre-decrement	High	Right-to-Left
+ (unary)	Unary plus	High	Right-to-Left
- (unary)	Unary minus	High	Right-to-Left
*	Multiplication	Medium	Left-to-Right
/	Division	Medium	Left-to-Right
%	Modulus	Medium	Left-to-Right
+	Addition	Low	Left-to-Right
-	Subtraction	Low	Left-to-Right

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Examples

- 3 + 5 * 2 evaluates as
  - 3 + (5 * 2), which is 3 + 10,
  - Resulting in 13.

- (3 + 5) * 2
  - Evaluates as 8 * 2,
  - which is 16.

- Examples

① `8 / 4 * 16`

`2 * 16`

`32`

② `8 * 4 / 16`

`32 / 16`

`2`

③ Assume a = 8, b = 15, c = 4.

`a * ((a%5) * (4 + (b-3)/(c+2)))`

`2 * ((8%5) * (4 + (15-3)/(4+2)))`

- Parentheses will have the highest priority and then next priority is for the operators.

`2 * ( 3 * (4 + (15-3)/(4+2)))`

`2 * (3 * (4 + 12 / (4+2)))`

`2 * (3 * (4 + 12/6))`

`2 * (3 * (4 + 2))`

`2 * (3 * 6)`

`2 * 18`

`36`

- Example

(4) Assume a=1, b=-5, c=6. Evaluate the following expression x1 = (-b+sqrt(b*b-4*a*c))/(2*a)

Soln: After Substituting

x1 = (-(-5) + sqrt((-5)*(-5)-4*1*6))/(2*1)

x1 = (5 + sqrt((-5)*(-5)-4*1*6))/(2*1)

x1 = (5 + sqrt(25 - 4*1*6))/(2*1)

x1 = (5 + sqrt(25 - 4*6))/(2*1)

x1 = (5 + sqrt(25 - 24))/(2*1)

x1 = (5 + sqrt(1))/(2*1)

x1 = (5 + 1.0)/(2*1)

x1 = 6.0/(2*1)

x1 = 6.0/2

x1 = 3.0

Relational Operators

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Relational operators are used to compare values in expressions

Description	Operator	Priority	Associativity
Less than	<	1	Left to Right
Less than/equal	<=	1	Left to Right
Greater than	>	1	Left to Right
Greater than/equal	>=	1	Left to Right
Equal	==	2	Left to Right
Not Equal	!=	2	Left to Right

Equality Operators

Rules to follow while evaluating Relational Expression

- Evaluate Expression within parenthesis
- Evaluate Unary operators
- Evaluate Arithmetic Expression
- Evaluate Relational Expression

Evaluate the following expression

100 /20< = 10 - 5 + 100 % 10 - 20 == 5 >= 1 ! = 20

- No expression inside parenthesis and no unary operators.
- Arithmetic operators are evaluated first next relational

5 < = 10 - 5 + 100 % 10 - 20 == 5 >= 1 ! = 20

5 < = 10 - 5 + 0 - 20 == 5 >= 1 ! = 20

5 < = 5 + 0 - 20 == 5 >= 1 ! = 20

5 < = 5 - 20 == 5 >= 1 ! = 20

5 < = -15 == 5 >= 1 ! = 20

0 == 5 >= 1 ! = 20

0 == 1 ! = 20

0 ! = 20

1

Result = 1

Logical Operators

- Logical operators are used to perform logical operations on boolean values.
- Essential for controlling the flow of a program, especially in conditional statements and loop
- Used to combine 2 or more relational expressions are called logical operations
- Expression contains only logical operators are called logical expression

Description	Operator	Priority	Associativity
NOT(Unary Operator)	!	1	Left to Right
AND(Binary)	&&	2	Left to Right
OR(Binary)	||	3	Left to Right

Logical NOT(!)

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Operand	!Operand
True(1)	False(0)
False(0)	True(1)

Logical AND(&&)

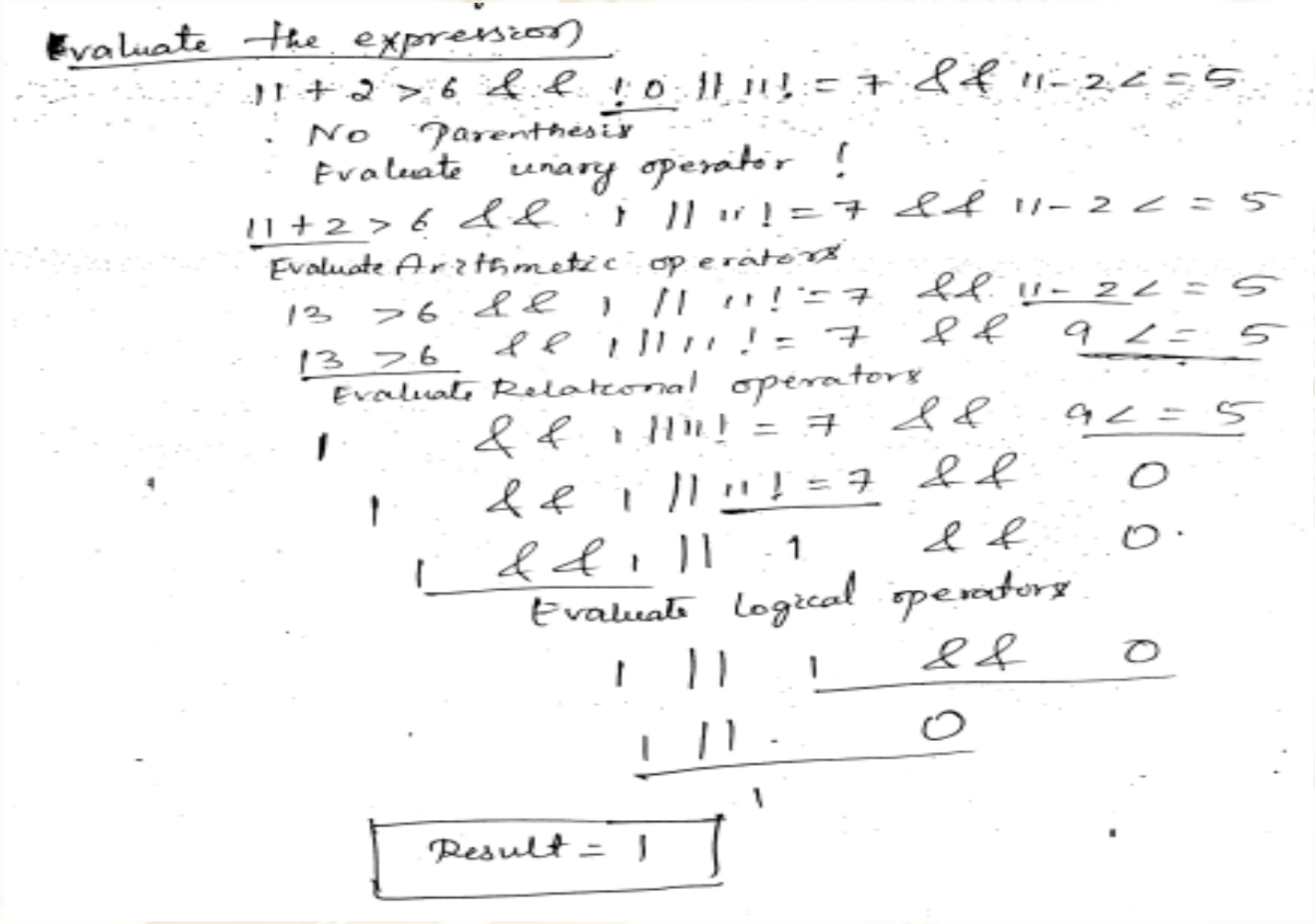
Operand1	AND	Operand2	Result
True(1)	&&	True(1)	True(1)
True(1)	&&	False(0)	False(0)
False(0)	&&	True(1)	False(0)
False(0)	&&	False(0)	False(0)

Logical OR(II)

Operand1	OR	Operand2	Result
True(1)	||	True(1)	True(1)
True(1)	||	False(0)	True(1)
False(0)	||	True(1)	True(1)
False(0)	||	False(0)	False(0)

Rules to be follow to evaluate Logical Expression

- Evaluate Expression within parenthesis
- Evaluate Unary operators
- Evaluate Arithmetic Expression
- Evaluate Relational Expression
- Evaluate Logical Expression



Conditional Operators.

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Make decisions based on conditions.
- Used within expressions to choose between two values based on a condition.
- The most common conditional operator is the ternary operator
- Ternary conditional operator is a compact way to express a simple if-else statement in a single line.
- It takes three operands and has the following syntax
  - condition ? expression_if_true : expression_if_false;
  - condition: An expression that evaluates to true or false.
  - expression_if_true: The expression that is evaluated and returned if the condition is true.
  - expression_if_false: The expression that is evaluated and returned if the condition is false.

Example

```
#include <stdio.h>
int main()
{
int a = 10, b = 20;
int max = (a > b) ? a : b; // max will be 20
printf("Max value is %d\n", max);
return 0;
}
```

Operator Precedence and Associativity

- **Precedence**:
  - The ternary operator (? :) has higher precedence than assignment operators but lower than most arithmetic operators.
- **Associativity**: Right-to-Left

- Program to find largest of 3 nos

```
#include <stdio.h>
int main()
{
int a, b, c, largest;
printf("Enter three numbers: ");
scanf("%d %d %d", &a, &b, &c);
largest=(a>b)?((a>c)?a:c)(b>c)?b:c);
printf("The largest number is= %d\n", largest);

    return 0;
}
```

Bitwise Operators

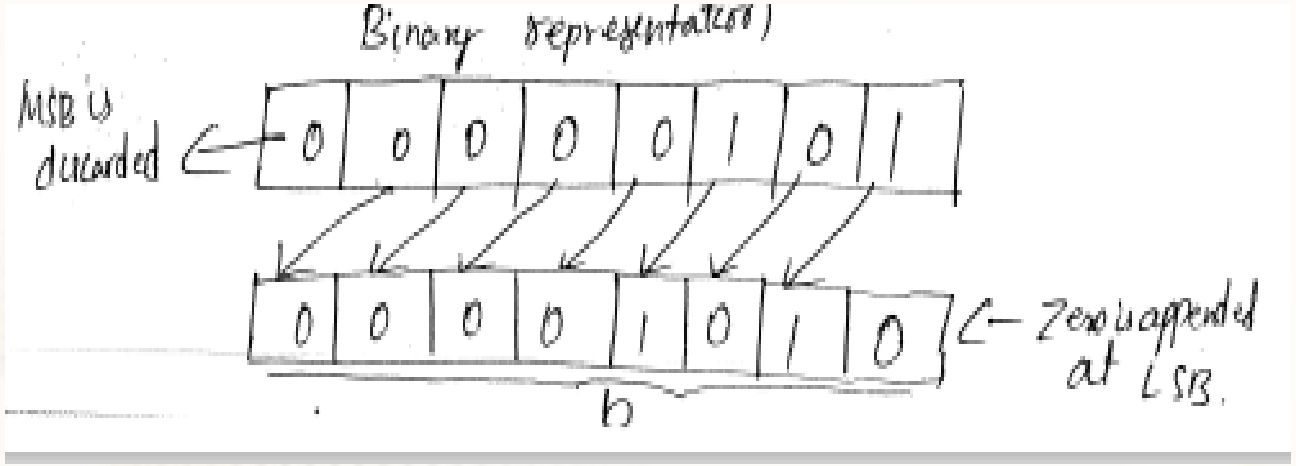
- Bitwise operators perform operations on binary representations of integers

Descriptions	Operator	Priority	Associativity
Bit-wise Negate	~	1	Left to right
Left Shift	<<	2	Left to right
Right Shift	>>	2	Left to right
Bit-wise and	&	3	Left to right
Bit-wise xor	^	4	Left to right
Bit-wise or	|	5	Left to right

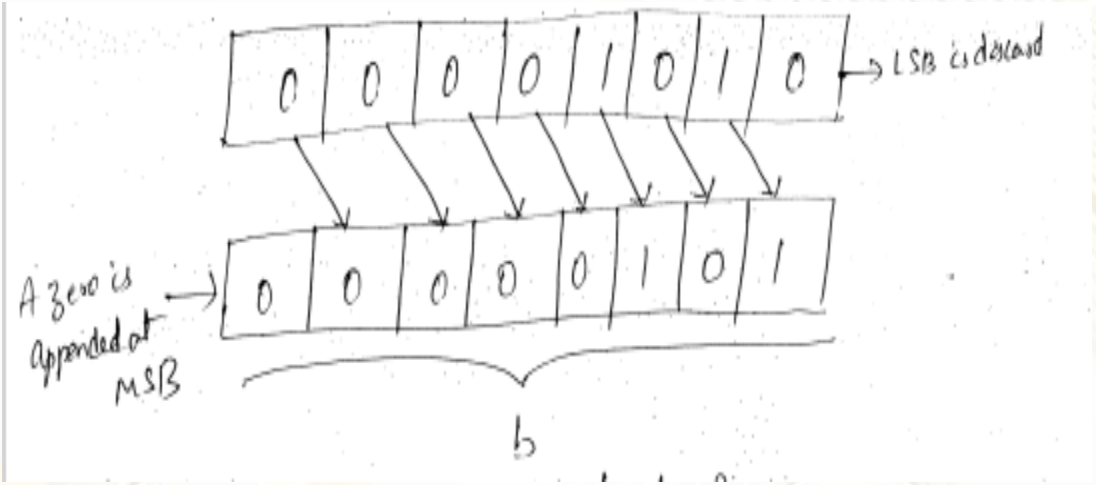
- Bit-wise Negate(~) (one's complement)
  - Inverts all the bits of the operand.
  - This means each 1 becomes 0, and each 0 becomes 1.
  - Example: ~5 --□ 0101 (5 in binary) , result□ 1010

Left and Right shift Operators

- Shifts the bits of the first operand to the left by the number of positions specified by the second operand and vice versa for right shift.
- The vacated positions on the right are filled with 0.
- Example: b=5 << 1



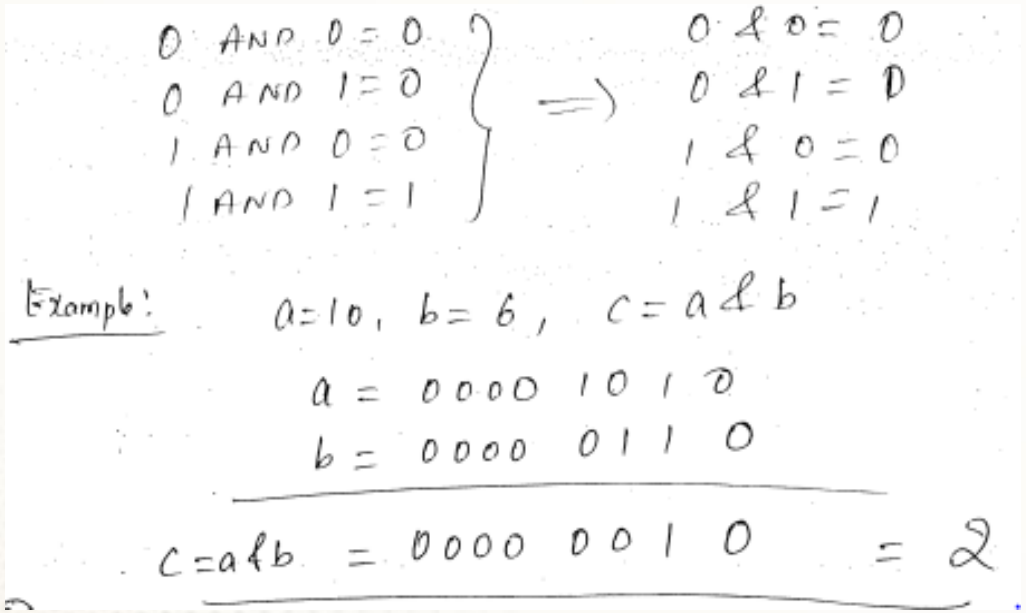
b=10>>1

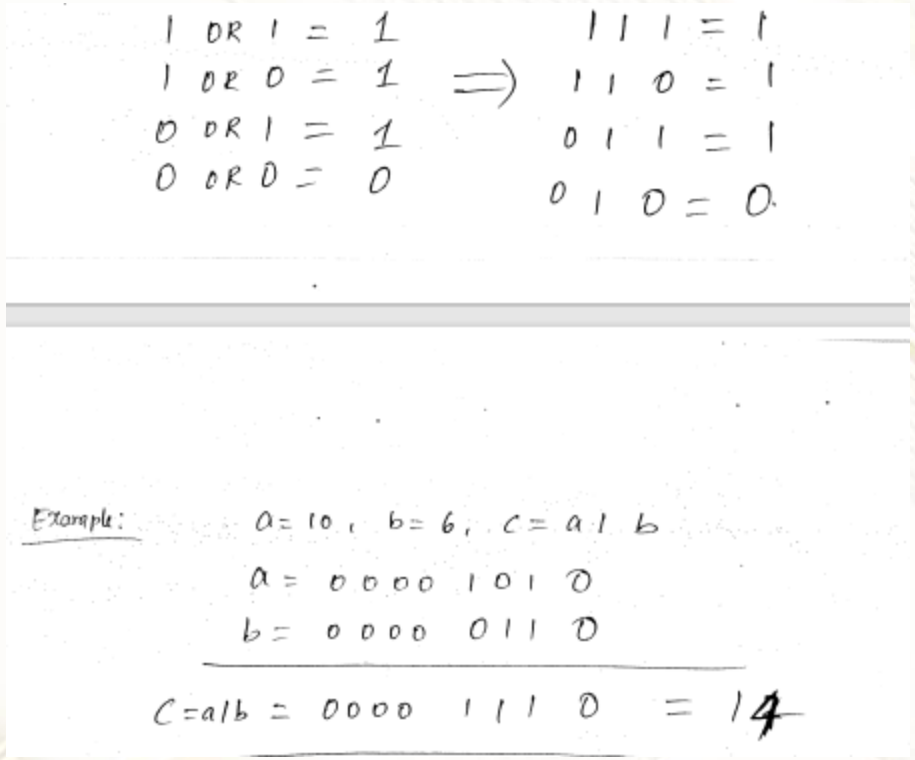


- Shift towards left by 1 bit is equivalent to multiplying by 2
- Shift towards Right by 1 bit is equivalent to divide by 2

Bit-wise And & Bit-wise Or

- Each bit in the result is 1 if the corresponding bits of both operands are 1,otherwise it is 0.





- Each bit in the result is 1 if at least one of the corresponding bits of the operands is 1

Bit-wise XOR

- It is represented by the caret symbol (^).
- It performs a bitwise exclusive OR operation between two integer values.

Example

```
#include <stdio.h>
int main() {
unsigned int a = 12;  // Binary: 00001100
unsigned int b = 7;   // Binary: 00000111
unsigned int result = a ^ b; // Binary result: 00001011
printf("a = %u (Binary: %b)\n", a, a);
printf("b = %u (Binary: %b)\n", b, b);
printf("a ^ b = %u (Binary: %b)\n", result, result);return 0;
}
```

A	B	A^B
0	0	0
0	1	1
1	0	1
1	1	0

Assignment Operators

- Assignment operators are used to assign values to variables.
- Most commonly used assignment operator is the basic =
- There are several compound assignment operators that combine an operation with an assignment
- Basic Assignment(=) Operator
  - Assigns the value on the right to the variable on the left.
  - Example: Int a=10;
- Compound Assignment Operators
  - These operators combine an arithmetic or bitwise operation with assignment

+=	-=	*=	/=	%=	&=	|=	^=
int a=10	int a=10	int a=10	int a=10	int a=10	int a=12, binary:00001100	int a=12, binary:00001100	int a=12, binary:00001100
a+=3 a=a+3	a-=3 a=a-3	a*=3 a=a*3	a/=3 a=a/3	a%=3 a=a%3	int b=7,binary:00000111	int b=7,binary:00000111	int b=7,binary:00000111
a=13	a=7	a=30	a=3	a=1	a &= b; Result: 00000100 (Binary), a now holds 4	a |= b; // Result: 00001111 (Binary), a now holds 15	a ^= b; // Result: 00001011 (Binary), a now

Cont..

- Basic Assignment: =
- Arithmetic Assignment: +=, -=, *=, /=, %=
- Bitwise Assignment: &=, |=, ^=, <<=, >>=

<<=	>>=
int a=5;	int a = 20;
Binary: 00000101	// Binary: 00010100
a <<= 1; // Result: 00001010 (Binary), a now holds 10	a >>= 2; // Result: 00000101 (Binary), a now holds 5

Type Casting and Type Conversion

- **Type conversion** and **type casting** are two related but distinct concepts used to handle and manipulate data type.
- **Type conversion** refers to the automatic or implicit conversion of data from one type to another by the compiler.
- This happens when the compiler converts data types as part of an expression evaluation to ensure the operation is performed correctly.
- Implicit Type Conversion
  - When performing arithmetic operations involving mixed types, the compiler automatically converts the operands to a common type, usually the type with the highest precision.
  - When assigning a value of a smaller data type to a larger data type, the conversion happens implicitly.

Ex: `int a = 5;  float b = 4.2;`

`float result = a + b;` // 'a' is implicitly converted to float // 'result' will be 9.2 (float)

Type Casting

- **Type casting** is a method used to explicitly convert a variable from one type to another.
- This is done using a cast operator and allows you to control the conversion process more precisely.
- **Syntax**: (target_type) expression
- Example:**Casting Between Numeric Types:** When you need to explicitly convert one numeric type to another, you use type casting.

```
int a = 5; float b = 2.3;

float result = (float) a / b; // Cast 'a' to float before division // 'result' will be 2.173913 (float)
```

Cont...

- **Casting to a Smaller Type:**
  - Casting from a larger to a smaller type may lead to loss of data.

Example

```
double a = 9.99;
int b = (int) a;  // Cast 'a' to int // 'b' will be 9, fractional part .99 is discarded
```


RV
UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS
Thank You